Orbiter Assignment Report

Algebraic Geometry over Finite Fields using Orbiter

Sudenaz Uçar

June 2026

Note: *In accordance with the assignment rules, the Orbiter guide (`https://www.math.colostate.edu/~betten/orbiter/users_guide.pdf`) and its example `makefile` were heavily consulted to find the correct command syntax for each problem. The methodology sections below demonstrate how the correct commands were discovered using the guide's examples.*

Problem 1 — GF(9) Cheat Sheet and Table Drawings

I have created an Orbiter cheat sheet for the finite field GF(9) and drown the addition and multiplication tables as colored matrix images using the `-draw_matrix` command.

To find how to generate a cheat sheet and draw the tables properly, I searched the Orbiter example `makefile`:

```
grep -n "cheat_sheet_GF\|draw_matrix" /Users/sudenazucar/Orbiter/...
```

From the results, I found that the cheat sheet requires the `-cheat_sheet_GF` and `-export_tables` activities. For drawing the matrix, I learned from the guide's examples that we need to define dummy vectors (`-repeat 1 9`) and use the `-partition 3 all_one all_one` syntax to format the 9x9 grid correctly.

Commands Used:

```
./orbiter.out -v 3 \
  -define F -finite_field -q 9 -end \
  -with F -do -finite_field_activity \
    -cheat_sheet_GF \
    -export_tables \
  -end

./orbiter.out -v 3 \
  -define all_one -vector -repeat 1 9 -end \
  -draw_matrix \
    -input_csv_file GF_q9_table_add.csv \
    -box_width 40 -bit_depth 24 \
    -partition 3 all_one all_one \
  -end \
```

```
  -draw_matrix \
    -input_csv_file GF_q9_table_mul.csv \
    -box_width 40 -bit_depth 24 \
    -partition 3 all_one all_one \
  -end
```

Results:

- The cheat sheet (`GF_9.tex`) shows that GF(9) has characteristic 3 and uses the primitive polynomial $x^2 + 1 \pmod 3$.
- The addition table is a perfect 9x9 Latin square.
- The multiplication table is a 9x9 grid where the first row and first column are all zeros (white color).

Problem 2 — Quartic Curve over GF(9)

I have retrieved a quartic curve (degree-4 plane curve) in PG(2,9) from the Orbiter knowledge base, createdd a detailed report, and determined the number of Kovalevski points.

To learn how to load a specific curve from the database and extract Kovalevski points, I checked the `makefile`:

```
grep -n "quartic_curve\|Kovalevski" /Users/sudenazucar/Orbiter/...
```

The guide showed that I must use `-catalogue 0` to load the first highly symmetric curve, and the activity `-export_something "Kovalevski_points"` to specifically extract these geometric features.

Command Used:

```
./orbiter.out -v 3 \
  -define F -finite_field -q 9 -end \
  -define P -projective_space -n 2 -field F -v 0 -end \
  -define C -quartic_curve -space P -catalogue 0 -end \
  -with C -do \
    -quartic_curve_activity \
      -report \
      -export_something "Kovalevski_points" \
    -end
```

Results and Analysis:
The equation of this curve is $X_1^4 + 6X_0^3X_2 + 4X_0X_2^3 = 0$.

- **Smoothness:** The curve has 0 singular points, meaning it is perfectly smooth.
- **Points:** There are exactly 28 points on the curve in PG(2,9).

- **Automorphism Group:** The symmetry group of the curve has an order of **12,096**. This group is isomorphic to PSU(3,3).
- **Kovalevski Points:** A Kovalevski point (or sextactic point) is where the osculating conic touches the curve with a contact order of at least 6. Orbiter calculated exactly **63 Kovalevski points**. Because of the high symmetry, all 63 points form a single orbit under the automorphism group.

Problem 3 — Cubic Surface $F(a, b, c, d)$ over GF(9)

I have created the cubic surface in PG(3,9) using the parameters $(a, b, c, d) = (6, 3, 4, 6)$. Create a report and identify its isomorphism type from the Orbiter catalogue.

To find the correct command to define and identify the surface, I searched through the Orbiter examples:

```
grep -n "Fabcd\|cubic_surface\|recognize" /Users/sudenazucar/...
sed -n '13330,13380p' /Users/sudenazucar/Orbiter/...
```

This investigation revealed that the surface parameters are entered using `-family_general_abcd 6 3 4 6` and the classification is performed using the `-identify_general_abcd` activity.

Commands Used:

```
./orbiter.out -v 3 \
    -define F -finite_field -q 9 -end \
    -define P -projective_space -n 3 -field F -v 0 -end \
    -define S -cubic_surface \
        -space P -family_general_abcd 6 3 4 6 \
    -end \
    -define Control -poset_classification_control -W \
        -problem_label "surf_q9" \
    -end \
    -define Classify -classify_cubic_surfaces \
        -use_double_sixes \
        -projective_space P \
        -poset_classification_control Control \
    -end \
    -define Orb -orbits \
        -on_cubic_surfaces Classify \
    -end \
    -with Orb -do \
        -cubic_surfaces_after_classification_activity \
        -identify_general_abcd \
    -end
```

Results and Analysis:

- **Points and Lines:** Orbiter found that the surface contains exactly **145 points** and **27 lines**. A classical rule states that smooth cubic surfaces always contain exactly 27 lines, which confirms our surface is smooth.
- **Identification:** The `-identify_general_abcd` command matched our surface with the catalogue. The output showed it is isomorphic to `iso_type 0`. This is the first (and most symmetric) isomorphism class in the GF(9) catalogue.
- **Transformation:** During the matching process, the parameters (6,3,4,6) were transformed to (8,7,6,5). This parameter change corresponds to applying a field automorphism in GF(9).

Problem 4 — Isomorphism Between Two Quartic Varieties

I have tested if the following two varieties are isomorphic in PG(2,9):

$$V_1 : X_0^4 + X_1^4 + X_2^4 = 0$$

$$V_2 : X_0^4 + X_1^4 + X_2^4 + X_0^3X_1 + 5X_0^3X_2 + X_0X_1^3 + 7X_1^3X_2 + 7X_0X_2^3 + 5X_1X_2^3 = 0$$

To learn how to test isomorphisms between custom varieties, I searched the `makefile` for testing commands:

```
grep -n "isomorphism_testing\|test_isomorphism" /Users/...
```

I found the `-test_isomorphism` command and learned how to input custom equations using the `-equation_by_coefficients` flag based on the standard monomial ordering.

Command Used:

```
./orbiter.out -v 6 \
    -define F -finite_field -q 9 -end \
    -define P -projective_space -n 2 -field F -v 0 -end \
    -define R -polynomial_ring \
        -field F -number_of_variables 3 -homogeneous_of_degree 4 \
        -monomial_ordering_partition -variables "X0,X1,X2" "X_0,X_1,X_2" \
    -end \
    -define V1 -variety -label_txt curve1 -projective_space P -ring R \
        -equation_by_coefficients "1,1,1,0,0,0,0,0,0,0,0,0,0,0,0" \
    -end \
    -define V2 -variety -label_txt curve2 -projective_space P -ring R \
        -equation_by_coefficients "1,1,1,1,5,1,7,7,5,0,0,0,0,0,0" \
    -end \
    -with V1 -and V2 -do -variety_activity \
        -test_isomorphism -nauty_control -nauty_log "nauty_log.txt" -end \
    -end
```

Results:

Orbiter successfully confirmed that V_1 **and** V_2 **are isomorphic**.

- Both varieties contain exactly **28 points** and have an automorphism group of order **12,096**.
- The explicit isomorphism matrix ϕ mapping V_1 to V_2 is:

$$\phi = \begin{bmatrix} 1 & 4 & 8 \\ 8 & 2 & 8 \\ 0 & 0 & 8 \end{bmatrix}$$

- The inverse matrix ϕ^{-1} mapping V_2 to V_1 is:

$$\phi^{-1} = \begin{bmatrix} 1 & 4 & 7 \\ 8 & 2 & 5 \\ 0 & 0 & 1 \end{bmatrix}$$

Problem 5 — Quartic Variety in PG(2,q) for q = 2, 4, 8

I have found the number of points and the automorphism group order for the variety:

$$X^4 + Y^4 + Z^4 + X^2Y^2 + X^2Z^2 + Y^2Z^2 + X^2YZ + XY^2Z + XYZ^2 = 0$$

over characteristic 2 fields (q = 2, 4, 8).

I needed to find how to compute the group order of a variety. Searching the guide's `makefile`:

```
grep -n "variety_activity.*automorph\|compute_group" ...
```

This search revealed the `-compute_set_stabilizer` command under `-variety_activity`, which successfully computes the automorphism group size of the points on the variety. The coefficient vector for our equation is `1,1,1,0,0,0,0,0,0,1,1,1,1,1,1`.

Results:
I ran the `-compute_set_stabilizer` activity for each field size:

Field Size (q)	Number of Points	Automorphism Group Order	Notes
q = 2	0	-	The variety is empty ove
q = 4	14	336	Group is isomorphic to P
q = 8	24	504	Group is isomorphic to P

Problem 6 — Classification of Cubic Curves in PG(2,4)

I have found a generating set of size 2 for the group PGL(3,4). Then, classify all cubic curves in PG(2,4) and find how many of them are nonsingular (smooth).

To find how to generate the group and classify curves, I searched the `makefile`:

```
grep -n "generating_set.*PGL\|orbits.*polynomials" ...
```

I found `-find_small_generating_set 2 100` to find the generators, and `-orbits -on_polynomials` to classify the curves.

Step 1: Generating Set for PGL(3,4)
Using the search command, Orbiter found two generators for PGL(3,4) (which has a group order of 60,480):

$$g_1 = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 2 \\ 2 & 0 & 0 \end{bmatrix}, \quad g_2 = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 0 & 1 \\ 3 & 1 & 1 \end{bmatrix}$$

Step 2: Orbit Classification
By running the `-orbits -on_polynomials` command, Orbiter grouped the 4^{10} possible polynomials into exactly **33 distinct orbits** (isomorphism classes).

Step 3: Finding Nonsingular Curves using Bash Scripting
A curve is nonsingular if its number of singular points is exactly 0. To check all 33 orbit representatives automatically, I wrote a bash script invoking Orbiter's `-singular_points` command:

```
for i in 0 3 9 10 11 12 18 19 20 24 34 44 50 51 55 175 176 ... 91990; do
  result=$(./orbiter.out -v 1 \
      -define F -finite_field -q 4 -end \
      -define P -projective_space -n 2 -field F -v 0 -end \
      -define R -polynomial_ring -field F -number_of_variables 3 ... -end \
      -define V -variety -projective_space P -ring R -equation_by_rank $i \
        -label_txt eqn_$i -label_tex eqn_$i -end \
      -with V -do -variety_activity -singular_points -end 2>&1 | \
        grep "number of singular points")
  echo "Rep $i: $result"
done
```

Results:
Exactly **20 out of the 33 orbits** returned `number of singular points = 0`. Therefore, there are exactly 20 nonsingular cubic curves in PG(2,4).

Problem 7 — Binary Hamming Graph of Order 7

I have created the binary Hamming graph $H(7, 2)$, find its diameter, computed its automorphism group order using Orbiter, and find its parameters as a distance-regular graph.

The official users guide mentions `-hamming`, but running this gave an `unrecognized option` error in Build 3352. To find the correct syntax, I searched the Orbiter C++ source code directly:

```
find /Users/sudenazucar/Orbiter -name "create_graph_description.cpp" ...
```

This source code investigation revealed that the updated syntax requires a capital letter (`-Hamming 7 2`) and the activity must be called using `-graph_theoretic_activity`.

Command Used:

```
./orbiter.out -v 3 \
  -define G -graph -Hamming 7 2 -end \
  -with G -do -graph_theoretic_activity \
    -automorphism_group \
  -end
```

Results from Terminal Output:
Orbiter triggered the internal `densenauty` graph library and printed the following critical results:

- `Gr->CG->nb_points = 128`: The graph has exactly $2^7 = \mathbf{128}$ **vertices**.
- The group order was successfully verified. Nauty output showed exactly **645,120**. This matches the theoretical hypercube group order $7! \times 2^7 = 5040 \times 128 = 645,120$.

Diameter and Distance-Regular Parameters:

- **Diameter:** The maximum distance is between a word and its inverse, so the **diameter is 7**.
- **Intersection Array:** As a distance-regular graph, the number of forward steps (b_i) and backward steps (c_i) at distance i are calculated mathematically:
  - $b_i = 7 - i \implies \{7, 6, 5, 4, 3, 2, 1\}$
  - $c_i = i \implies \{1, 2, 3, 4, 5, 6, 7\}$
  - **Intersection Array:** $\{7, 6, 5, 4, 3, 2, 1 \ ; \ 1, 2, 3, 4, 5, 6, 7\}$

Appendix: Files to Attach for the Instructor

- **Problem 1:** `GF_9.pdf` (The Cheat Sheet PDF) and the generated images `GF_q9_table_add_draw.b` & `GF_q9_table_mul_draw.bmp`.
- **Problem 2:** `quartic_curve_catalogue_q9_iso0_report.pdf`
- **Problem 4:** `nauty_log.txt` (Contains the Nauty processing details).
- **Problem 5:** `variety_curve_q4_report.pdf` & `variety_curve_q8_report.pdf`
- **Problem 6:** `poly_orbits_d3_n2_q4.pdf` & `poly_orbits_d3_n2_q4.csv`
- **Problem 7:** `Hamming_7_2_report.tex` .